Prepare o projeto e a API

Mesmo formato da Sprint 4: comandos por terminal, ações na tela, falas e resultados. Faça esta preparação antes de iniciar a gravação.

EQUIPE / REPOSITÓRIO

Abra a pasta no VS Code

João Viviani Baldini - RM 558596
Giuliano Ferreira Venceslau - RM 558674
Enrico Nikolay Meirelles Zeronian - RM 558557

Clone somente se ainda não tiver a pasta. Abra o VS Code e use **Terminal > Novo Terminal**. O README contém os requisitos de execução e os limites da demonstração.

```
git clone https://github.com/joao-baldini/GS2-Advanced-Programming-Mobile-Dev.git
cd GS2-Advanced-Programming-Mobile-Dev
code .
```

TERMINAL 1 / BACKEND

Spring Boot na porta 8080

Na raiz do projeto, entre em **backend/**. Java e javac devem ser **17**; o Maven deve usar esse JDK. Aguarde **Started UrbanRiskApplication** e mantenha o terminal aberto.

O H2 em arquivo guarda os registros em **backend/data/**. Execute sempre nessa pasta para reencontrar o mesmo banco após o reinício.

```
cd backend
java -version
javac -version
mvn -version
mvn spring-boot:run
```

TERMINAL 2 / API

Confirme o JSON antes de abrir o app

Abra outro terminal na raiz. O health responde **UP**. A lista retorna um objeto com **items**, **total** e horário do servidor. O primeiro banco recebe oito registros simulados; o total pode crescer durante o ensaio.

```
curl.exe http://localhost:8080/actuator/health
curl.exe "http://localhost:8080/api/events?size=3"
curl.exe http://localhost:8080/api/dashboard
```

Prepare o Expo e o celular

Instale as dependências antes de gravar. O app usa Expo SDK 58; a web é a prévia do mesmo código React Native, útil para ensaiar.

TERMINAL 3 / PRÉVIA WEB

Abra o app na porta 8081

Em outro terminal na raiz, entre em **frontend/**. Use **Node.js 24**. Abra **http://localhost:8081**, espere o status **Conectado** e deixe a primeira consulta terminar.

```
cd frontend
node --version
npm ci
npm run web
```

TERMINAL 3 / TELEFONE FÍSICO

Expo Go e computador na mesma rede

Para mostrar o app mobile, use os comandos abaixo em vez da inicialização web. Substitua **SEU_IPV4** pelo IPv4 do computador mostrado por **ipconfig**. O marcador não deve ser copiado literalmente.

No telefone, teste **http://SEU_IPV4:8080/api/events** no navegador. Depois leia o QR Code com **Expo Go compatível com SDK 58**. No iPhone, use a câmera. Se não conectar, confira IP, Wi-Fi e firewall da porta 8080.

```
cd frontend
ipconfig
$env:EXPO_PUBLIC_API_URL="http://SEU_IPV4:8080"
npx expo start --clear
```

ANTES DE GRAVAR

Deixe a demonstração pronta

Janelas: README, Terminal 1 (Backend), Terminal 2 (API), Terminal 3 (Expo) e tela do app. Se usar telefone, prepare uma captura da sua tela junto ao computador.

App: primeiro carregamento concluído; filtro **Todos os riscos**; busca vazia. O painel consulta a cada 5 segundos. O valor em ms mede a consulta, não esse intervalo.

Ensaio: confirme que consegue parar só a API, manter o Expo e reiniciar na mesma pasta. A gravação proposta dura cerca de 5 minutos; o enunciado não fixa duração.

Vídeo: proposta, API e análise

Trecho planejado: 0:00 a 1:45. Mostre a integração e explique a prioridade antes de cadastrar um novo relato.

CENA 1 / 0:00-0:30

Projeto e equipe

Na tela: No GitHub, mostre README, os três integrantes/RMs e as pastas backend/ e frontend/.

Fala sugerida: 'Esta é a Urban Risk Intelligence Platform. Reunimos sensores, bases públicas e relatos comunitários para acompanhar alagamentos, calor e interrupções de serviços. A API é Spring Boot e o app é React Native.'

Resultado esperado: Tema, equipe e duas stacks ficam claros. Não instale dependências durante a gravação.

CENA 2 / 0:30-1:05

API ativa e registros persistidos

Na tela: Mostre o Terminal 1 ativo. No Terminal 2, consulte eventos e destaque ID, source, sourceName, confidence, simulated e analysis.

Fala sugerida: 'A API persiste os registros no H2 em arquivo. Cada dado tem origem e confiança estimada. Os registros iniciais são simulados e identificados. A lista é ordenada por prioridade no servidor.'

Resultado esperado: JSON real da API. Selecione um ID existente para comparar com o aplicativo.

```
curl.exe "http://localhost:8080/api/events?size=2"
```

CENA 3 / 1:05-1:45

Painel e prioridade explicável

Na tela: No app conectado, compare um título/ID com a API. Abra o detalhe de um sensor e mostre fonte, confiança, análise de prioridade e recomendação.

Fala sugerida: 'O app consulta a API a cada cinco segundos. A análise combina severidade, confiança, impacto estimado e recência. Cada pontuação tem explicação: é apoio à triagem, não uma previsão treinada de IA.'

Resultado esperado: Os quatro fatores aparecem no detalhe. Pontuações antigas podem variar pela recência; a confiança é uma hipótese por fonte.

Vídeo: cadastro e novo sensor

Trecho planejado: 1:45 a 3:15. Cadastre pelo aplicativo e, depois, envie uma leitura para demonstrar a atualização automática.

CENA 4 / 1:45-2:35

Registro comunitário pelo aplicativo

Na tela: Toque em Registrar ocorrência. Tipo: Alagamentos; título: 'Alagamento - apresentação GS'; bairro: Ipiranga; descrição: 'Água acumulada na via após chuva. Registro acadêmico de demonstração.'; severidade: Alto; pessoas: 120. Registre, abra o detalhe e compare o ID com a API.

Fala sugerida: 'O relato é enviado pelo aplicativo, recebe ID do backend e fica persistido. Como é comunitário, entra com 60% de confiança estimada e precisa de verificação. Não pedimos nome nem documento.'

Resultado esperado: O mesmo título e ID no app e no JSON. Use a busca em Ocorrências para localizar o cadastro.

```
curl.exe "http://localhost:8080/api/events?type=FLOOD&neighborhood=Ipiranga"
```

CENA 5 / 2:35-3:15

Leitura simulada e alerta automático

Na tela: Volte a Visão geral, limpe a busca e selecione Todos os riscos. Envie o POST no Terminal 2. Não atualize manualmente. Aguarde a consulta e abra a Central de alertas.

Fala sugerida: 'Enviei uma leitura simulada de sensor à API. O app recebeu o evento na próxima consulta e o colocou na central por sua prioridade. É atualização periódica de cinco segundos.'

Resultado esperado: Novo 'Alagamento detectado pelo sensor', origem SENSOR, simulação identificada e prioridade 98 quando recente. Um novo evento deve aparecer na central.

```
curl.exe -X POST http://localhost:8080/api/demo/reading
```

NOTIFICAÇÕES / DEMONSTRAÇÃO

O que dizer com precisão

Primeira consulta: estabelece a referência; não notifica os eventos antigos. Envie o sensor depois de o app estar sincronizado.

Celular: a central oferece Ativar notificações. As notificações são locais, com o app em execução. Na web, os alertas ficam dentro do aplicativo; não há push remoto em segundo plano.

Simulador: só existe com modo demo ativo e permite uma leitura a cada 5 segundos. Resposta 429 significa aguardar antes de repetir.

Vídeo: queda e reconexão

Trecho planejado: 3:15 a 5:00. Esta é a comprovação central de disponibilidade pedida no enunciado: pare o backend de verdade e mostre a recuperação.

CENA 6 / 3:15-3:55

Pare somente o backend

Na tela: Deixe o app no topo de Visão geral. No Terminal 1, pressione Ctrl+C e espere encerrar. Mantenha Expo e app abertos. Aguarde até cerca de 8 segundos; mostre Desconectado, o aviso e o horário dos dados salvos. Navegue para provar que não travou.

Fala sugerida: 'Parei apenas o backend. O app percebe a falha e identifica que os dados são da consulta anterior. A interface continua funcionando, e o cache não é apresentado como nova leitura.'

Resultado esperado: Aviso de falha real e navegação preservada. O GET opcional abaixo deve falhar.

```
curl.exe --max-time 3 http://localhost:8080/api/events
```

CENA 7 / 3:55-4:40

Reinicie e acompanhe a recuperação

Na tela: No Terminal 1, dentro de backend/, execute o comando. Após Started UrbanRiskApplication, mantenha a câmera no app: ele mostra Conexão restabelecida por alguns segundos e volta a Conectado. Compare o ID do cadastro da cena 4.

Fala sugerida: 'Com a API de volta, o app reconecta automaticamente. O registro continua com o mesmo ID porque o banco é persistente. Não precisei reiniciar o aplicativo.'

Resultado esperado: Recuperação automática e dados preservados. Se acelerar o tempo de boot no vídeo, identifique o corte.

```
mvn spring-boot:run
```

CENA 8 / 4:40-5:00

Finalize e prepare a entrega

Na tela: Mostre Fontes de dados, os testes no README e encerre no painel conectado. Depois de gravar, publique o vídeo e acrescente a URL verdadeira no README.

Fala sugerida: 'Organizamos três origens com confiança e prioridade explicáveis. Validamos API, cliente HTTP e recuperação da conexão. As duas stacks e instruções estão no repositório. A demonstração usa dados simulados.'

Resultado esperado: Envie ENTREGA_GS.txt ao portal FIAP até 19/11/2026 às 23h55. Se usar YouTube, prefira Não listado. O roteiro Markdown no repositório inclui diagnóstico de problemas.